

TEMA 3. AUTOMATAS FINITOS

- Autómatas finitos deterministas.
- Autómatas finitos no deterministas.
- Autómatas finitos no deterministas con transiciones nulas.
- Autómatas probabilísticos.
- Análisis léxico.

AUTÓMATAS FINITOS DETERMINISTAS

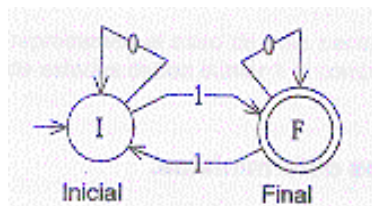
1. Autómatas finitos deterministas

Llamamos Autómata finito determinista a la quintupla formada por (E, Q, f, q_0, F) .

Es una máquina secuencial de Moore cuyo alfabeto de entrada es E y el alfabeto de salida es $\{0, 1\}$, y estando formado F por todos aquellos estados para los que se cumpla que $g(q)=1$.

2. Tablas y diagramas de transición

$f: Q \times E$	0	1
$\rightarrow I$	I	F
$*F$	F	I



3. Extensión de f a palabras

- $f(q, \xi) = q \quad \forall q \in Q$
- $f(q, ax) = f(f(q, a), x) \quad \forall a \in E, \forall x \in E^*, \forall q \in Q$

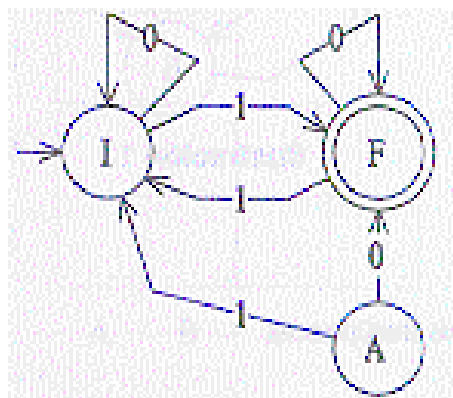
4. Lenguaje asociado a un Autómata finito determinista

Sea el autómata finito determinista $A = (E, Q, f, q_0, F)$. Decimos que $x \in E^*$ es aceptada por el autómata A si $f(q_0, x) \in F$.

Llamamos lenguaje asociado al autómata finito determinista A al conjunto de todas las palabras aceptadas por el autómata.

5. Estados accesibles

Un estado p es accesible si $\exists x \in E^*$ t.q. $f(q, x) = p$. Si todos los estados son accesibles decimos que dicho autómata es conexo.



6. Equivalencia de autómatas

Dos autómatas son equivalentes si reconocen el mismo lenguaje.

Dos estados p y q son equivalentes sii

$$f(p,x) = f(q,x) \quad \forall x \in E^*$$

7. Minimización de Autómatas

Construiremos un conjunto cociente de estados formado por las clases de equivalencia constituidas por estados equivalentes.

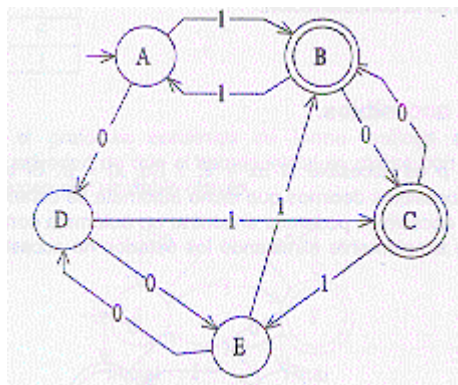
7.1. Algoritmo 1

1. Eliminar los estados inaccesibles.
2. Agrupar en una clase de equivalencia los estados finales y en otra el resto de estados.
3. Comprobar a qué clase de equivalencia pertenece el estado siguiente de todos los estados de una misma clase de equivalencia:
 - 3.1. Si son iguales esa es una clase de equivalencia.
 - 3.2. Si no son iguales comprobamos si existen otros estados dentro de la misma clase que se comportan igual y creamos una nueva clase.
4. Volver al paso 3 hasta que no haya más cambios.

7.2. Algoritmo 2

1. Eliminar los estados inaccesibles.
2. Crear una tabla en la cual las filas se nombran de arriba hacia abajo con los estados del autómata de manera ordenada empezando por el segundo estado, y las columnas se nombran igualmente, pero empezando por el primer estado y omitiendo en esta ocasión el último. Únicamente se utilizará la semitabla situada por debajo de la diagonal, incluyendo dicha diagonal.
3. Marcar todas aquellas celdas formadas por un estado final y otro no final.
4. Para todas las celdas no marcadas comprobar $f(q_i, e_k)$, $f(q_j, e_k)$.
 - 4.1. Si son iguales no hacer nada.
 - 4.2. Si está marcada la celda $(f(q_i, e_k), f(q_j, e_k))$ marcar la celda (q_i, q_j) .
 - 4.3. Si no está marcada añadir la referencia (q_i, q_j) a la celda $(f(q_i, e_k), f(q_j, e_k))$.
5. Si al marcar una casilla existen referencias a otras casillas, marcar todas las casillas referenciadas.
6. Cada estado será equivalente a todos los que no estén marcados en su columna y en su fila.

Ejemplo:



8. Autómata finito determinista asociado a una gramática regular

Sea la gramática tipo 3 recursiva a izquierdas:

$$G = (V, T, P, S).$$

con P de la forma:

$$S \rightarrow \xi, A \rightarrow Bu, A \rightarrow u, \quad u \in T, \quad A, B \in V$$

El autómata finito determinista asociado será de la forma:

$$A = (E, Q, f, q_0, F)$$

donde:

- $E = T$
- $Q = V \cup \{p, q\}$
- $q_0 = p$
- $F = \{S\}$

siendo q el *estado muerto* del autómata.

La función f de transición quedaría definida como sigue:

- $f(U, t) = V$ si $V \rightarrow Ut \in P$
- $f(p, t) = V$ si $V \rightarrow t \in P$
- $f(U, t) = q \quad \forall t \in T \text{ t.q. } V \rightarrow Ut \notin P$
- $f(p, t) = q \quad \forall t \in T \text{ t.q. } V \rightarrow t \notin P$
- $f(q, t) = q \quad \forall t \in T$
- Si $S \rightarrow \xi \in P$ entonces $p \in F$.

Ejemplo: Construir el A.F.D. asociado a la gramática:

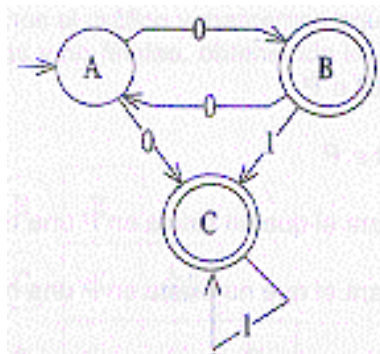
$$G = (\{0,1\}, \{S,U,V\}, \{S \rightarrow U0|V0, U \rightarrow S1|1, V \rightarrow S0|0\}, S)$$

AUTÓMATAS FINITOS NO DETERMINISTAS

1. Autómatas Finitos No Deterministas

Un autómata finito es no determinista si se cumple al menos una de las siguientes condiciones:

- a) No tiene definidas todas las transiciones.
- b) Al menos una de sus transiciones es múltiple.



2. Extensión de f a palabras

Dada $f: Q \times E \rightarrow P(Q)$, la extendemos a palabras de la siguiente manera:

$$f': Q \times E^* \rightarrow P(Q)$$

calculándose f' como sigue:

- $f'(p, \xi) = \{p\}$
- $f'(p, ax) = \bigcup_{q \in f(p,a)} f'(q,x)$

Ejemplo: A.F.N.D. que reconozca la subcadena 010010 (cabecera de un mensaje) y determinar $f'(q_0, 010)$.

3. Lenguaje aceptado por un A.F.N.D.

Una palabra es aceptada por un A.F.N.D. si al introducir dicha palabra en el autómata a partir del estado inicial es posible llegar a un estado final mediante alguno de los caminos posibles.

El lenguaje aceptado por un A.F.N.D. será el formado por el conjunto de palabras aceptadas por el mismo:

$$L(A) = \{x \in E^* \mid f'(q_0, x) \cap F \neq \emptyset\}$$

Ejemplo: Construir un A.F.N.D. que reconozca las constantes reales, las cuales pueden expresarse de la siguiente manera: $[+|-](0..9)^+[(0..9)^*[E[+|-](0..9)^+]$

4. Teorema

Un lenguaje es aceptado por un A.F.D. $\Leftrightarrow$ es aceptado por un A.F.N.D.

Demostración

$\Rightarrow$ Trivial, ya que todo A.F.D. es un A.F.N.D.

$\Leftarrow$ Ver apartado siguiente.

5. Paso de A.F.N.D. a A.F.D.

1. Se crea $P(Q)$ y cada elemento de este nuevo conjunto será un estado del A.F.D. Si una componente de un elemento de $P(Q)$ es un estado final, dicho elemento será también un estado final.
2. Añadir un nuevo estado w al que denominaremos *estado muerto*.
3. Las entradas serán las mismas.
4. Si en la tabla de transición del A.F.N.D. un estado pasa con una entrada a varios estados, la combinación de éstos pertenece a $P(Q)$, rellenando la casilla correspondiente de la tabla de transición del A.F.D. con dicho elemento de $P(Q)$.
5. Para rellenar las casillas de los elementos compuestos de $P(Q)$ calculo todos los estados siguientes de cada uno de los estados componentes de cada uno de dichos elementos de $P(Q)$. El conjunto de todos estos estados siguientes conforman otro elemento de $P(Q)$, procediendo a su anotación en la tabla de transición del A.F.D.
6. Cualquier casilla que quede vacía la relleno con el estado w .
7. Las casillas del estado w las relleno con el mismo estado w .

Ejemplo: Obtener el A.F.D. asociado

	0	1
→ A	B, C	
*B	A	C
*C		C

Ejemplo: Obtener el A.F.D. asociado

$G = (\{0, 1\}, \{S, U, V\}, \{S \rightarrow U0|V1, U \rightarrow S1|1, V \rightarrow S1|0\}, S)$

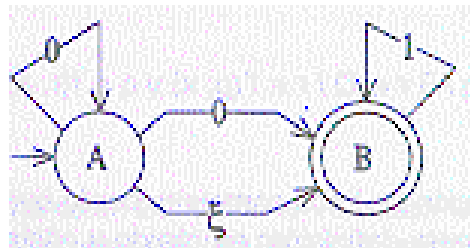
AUTOMATAS FINITOS NO DETERMINISTAS CON TRANSICIONES NULAS

1. A.F.N.D. con Transiciones Nulas

Son A.F.N.D. que pueden pasar de un estado a otro sin consumir ninguna entrada (transición nula).

La función de transición quedaría de la siguiente manera:

$$f: Q \times (E \cup \{\xi\}) \rightarrow P(Q)$$



Ejemplo: ξ -A.F.N.D. para $L = \{0^i 1^j 2^k \mid i, j, k \geq 0\}$

Ejemplo: ξ -A.F.N.D. para $L = \{a^i b^{2j} c^k \mid j \geq 0, i, k = 0, 1\}$

2. Clausura

- *Clausura de un estado p :* conjunto formado por todos los estados a los que se puede llegar a partir de p mediante transiciones nulas:

$$Cl(p) = \{q \mid \exists q_1, \dots, q_n, q_1 = p, q_n = q, (q_i \in f(q_{i-1}, \xi), \forall i = 2, \dots, n)\}$$

- *Clausura de un conjunto de estados:* conjunto de estados formado por las clausuras de todos los estados del conjunto.

3. Extensión de f a palabras

Definimos:

$$f(R,a) = \cup_{q \in R} f(q,a)$$

Definimos también:

$$\begin{aligned} f' : Q \times E^* &\rightarrow P(Q) \\ (q,u) &\rightarrow H \end{aligned}$$

de forma recursiva:

$$\begin{aligned} f'(q,\xi) &= Cl(q) \\ f'(q,au) &= f'(f(Cl(q),a),u) \end{aligned}$$

definimos f' para un conjunto de estados:

$$f'(q,au) = \cup_{p \in f(Cl(q),a)} f'(p,u)$$

4. Palabra aceptada por un ξ -A.F.N.D.

Dado $A=(E, Q, f, q_0, F)$, $u \in E^*$ es aceptada por $A \Leftrightarrow f'(q_0, u) \cap F \neq \emptyset$

Ejemplo: A partir del ξ -A.F.N.D. reconocedor del lenguaje $L = \{a^i b^{2j} c^k \mid j \geq 0, i, k = 0, 1\}$, determinar si la palabra abb es aceptada por el lenguaje.

5. Lenguaje aceptado por un ξ -A.F.N.D.

$$L(A) = \{u \in E^* \mid f'(q_0, u) \cap F \neq \emptyset\}$$

Ejemplo: Construir un ξ -A.F.N.D. que reconozca palabras que contienen la secuencia 010 ó 111.

Ejemplo: Construir un ξ -A.F.N.D. que reconozca palabras que contienen la secuencia 010 y 111, en dicho orden.

6. Teorema

Un lenguaje es aceptado por un ξ -A.F.N.D. $\Leftrightarrow$ es aceptado por un A.F.N.D.

Demostración

$\Leftarrow$ Trivial, ya que todo A.F.N.D. es un ξ -A.F.N.D.

$\Rightarrow$ Ver algoritmo apartado siguiente.

7. Algoritmo de paso de ξ -A.F.N.D. a A.F.N.D.

Dado un lenguaje L aceptado por el ξ -A.F.N.D.:

$$A = (E, Q, f, q_0, F)$$

el A.F.N.D. que reconoce dicho lenguaje se construye de la siguiente manera:

$$A' = (E, Q, g, q_0, F')$$

donde:

$$F' = \begin{cases} F & \text{si } Cl(q_0) \cap F = \emptyset \\ F \cup \{q_0\} & \text{si } Cl(q_0) \cap F \neq \emptyset \end{cases}$$

y

$$g(q, a) = f'(q, a)$$

8. Algoritmo alternativo de paso de λ -A.F.N.D. a A.F.N.D.

1. Calcular la clausura de todos los estados del λ -A.F.N.D. que tienen transiciones nulas.
2. Copiar la tabla de transición excepto la columna correspondiente a la cadena vacía.
3. Para cada uno de los estados del λ -A.F.N.D. copiar en su fila las filas de los estados de su clausura.
4. Si en la clausura de algún estado hay algún estado final incluir dicho estado dentro del conjunto de los estados finales.

Ejemplo: A partir del λ -A.F.N.D. asociado al lenguaje $L = \{0^i 1^j 2^k \mid i, j, k \geq 0\}$, obtener el A.F.N.D. correspondiente.

9. Teorema

Para toda gramática tipo 3 existe un A.F.D. que acepta el lenguaje aceptado por la gramática.

Demostración

1. *Generación de un λ -A.F.N.D. a partir de una gramática lineal por la derecha* → ver apartado 10
2. *Generación de un λ -A.F.N.D. a partir de una gramática lineal por la izquierda* → ver apartado 11

Finalmente, bastará con aplicar los algoritmos vistos anteriormente para pasar un λ -A.F.N.D. a A.F.N.D., y éste a A.F.D.

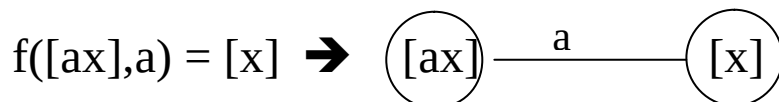
10. Generación de un ξ -A.F.N.D. a partir de una gramática lineal por la derecha

Dada una gramática de tipo 3 lineal por la derecha $G = (V, T, P, S)$, construimos el ξ -A.F.N.D. asociado $A = (E, Q, f, q_0, F)$ de la siguiente manera:

- $E = T$
- $Q = \{[x] \mid x = S \text{ ó } (\exists A \rightarrow ux \in P)\}$
- $F = \{[\xi]\}$
- $q_0 = [S]$

Para todas las producciones añadir en el autómata una transición nula desde el estado correspondiente a la parte izquierda de la producción hasta el estado correspondiente a la parte derecha de la misma.

Añadir todas las transiciones posibles del tipo:



Ejemplo: Construir el ξ -A.F.N.D. asociado a la siguiente gramática $G = (\{S, A, C\}, \{0, 1\}, \{S \rightarrow 0A, A \rightarrow 10A \mid \xi, C \rightarrow 01\}, S)$

11. Generación de un λ -A.F.N.D. a partir de una gramática lineal por la izquierda

1. Invertir la gramática G invirtiendo la parte derecha de las producciones, obteniendo una gramática G' lineal por la derecha $\rightarrow L(G) = L(G')^{-1}$
2. Construimos el λ -A.F.N.D. A' asociado a $G' \rightarrow L(A') = L(G') = L(G)^{-1}$
3. Invertimos el λ -A.F.N.D. A' obteniendo un λ -A.F.N.D. $A \rightarrow L(A) = L(A')^{-1} = L(G')^{-1} = L(G)$:
 - 3.1. Cambiando el sentido de las flechas.
 - 3.2. Cambiando el estado inicial y final.

Ejemplo: Obtener un λ -A.F.N.D. asociado a la gramática $G = (\{S\}, \{0,1\}, \{S \rightarrow S10 \mid 0\}, S)$.

12. Teorema

Si un lenguaje es aceptado por un A.F.D. entonces es generado por una gramática lineal por la derecha y por otra lineal por la izquierda.

Demostración

1. *Generación de una gramática lineal por la derecha a partir de un A.F.D.* $\rightarrow$ ver apartado 13
2. *Generación de una gramática lineal por la izquierda a partir de un A.F.D.* $\rightarrow$ ver apartado 14

13. Generación de una gramática lineal por la derecha a partir de un A.F.D.

Dado un A.F.D. $A = (E, Q, f, q_0, F)$, obtenemos la gramática lineal por la derecha asociada $G = (V, T, P, S)$ de la siguiente manera:

- $V = Q$
- $T = E$
- $S = q_0$

Las producciones asociadas a la gramática (*Bien Formada*) las obtenemos del siguiente modo:

- Si $f(q_i, a) = q_j$ entonces $q_i \rightarrow aq_j \in P$
- Si $f(q_i, a) = q_j$ y $q_j \in F$ entonces $q_i \rightarrow a \in P$
- Si $q_0 \in F$ entonces $q_0 \rightarrow \xi$

14. Generación de una gramática lineal por la izquierda a partir de un A.F.D.

1. Invertir el A.F.D.
2. Calcular la gramática lineal por la derecha asociada al A.F.D. invertido.
3. Invertir la gramática.

Ejemplo: Obtener una gramática lineal por la derecha y otra lineal por la izquierda asociada al autómata $A = (\{0, 1\}, \{Q_1, Q_2, Q_3\}, f, Q_1, \{Q_2, Q_3\})$, siendo f :

f	0	1
$\rightarrow Q_1$	Q_2	Q_3
$*Q_2$	Q_1	Q_3
$*Q_3$	Q_2	Q_2

AUTÓMATAS PROBABILÍSTICOS

1. Definición

Llamamos autómata probabilístico a la quintupla $(E, Q, M, P(0), F)$, donde:

- E : alfabeto de entrada.
- Q : conjunto de estados.
- M : matrices de probabilidad de transición.
- $P(0)$: vector probabilístico de estado inicial.
- F : conjunto de estados finales.

2. Matrices de probabilidad de transición

El conjunto M tendrá tantas matrices como símbolos haya en el conjunto E de entrada y tendrán tantas filas y columnas como estados haya en el conjunto Q .

La celda (i, j) de una matriz de probabilidad M asociada a un símbolo de entrada a , indica la probabilidad de que estando en el estado p_i pasemos al estado p_j leyendo una a .

$$M(a) = \begin{array}{c|cc} & p & q \\ \hline p & 0.25 & 0.75 \\ q & 0.6 & 0.4 \end{array}$$

Por tanto, la suma de todas las celdas de una fila deberá ser igual a 1 (es la probabilidad de que, estando en un estado dado, se avance a cualquier otro estado, al leer el símbolo asociado a la matriz de probabilidad M actual).

3. Vector de estado

Es el vector $P(t) = (P_0(t), P_1(t), \dots, P_n(t))$, donde $P_i(t)$ es la probabilidad de que el autómata se encuentre en el estado q_i en el instante t .

Debe cumplirse que:

$$\sum_{0 \leq i \leq n} P_i(t) = 1$$

El autómata comienza con un vector de estado inicial ($t=0$) y se recibe una palabra $z=ab\dots n$. La probabilidad de que el autómata se encuentre en el estado q_i después de leer la palabra z será:

$$P(z) = P(0) \times M(a) \times M(b) \times \dots \times M(n)$$

4. Lenguaje reconocido por un autómata probabilístico

Ampliamos el concepto de autómata probabilístico a la séxtupla $(E, Q, M, P(0), F, u)$, donde u es un valor entre 0 y 1 llamado *punto de corte* o *umbral*.

Una palabra x es aceptada por un autómata probabilístico si $P_i(x) \geq u$, con $q_i \in F$.

Llamamos lenguaje aceptado por un autómata probabilístico al conjunto de palabras aceptadas por el mismo.

Para calcular si una palabra es aceptada por un autómata probabilístico basta con multiplicar el vector de estado final de dicha palabra por una matriz de dimensión $n \times 1$, siendo n el número de estados del autómata, valiendo uno cada una de las filas de los estados finales y cero el resto.

Ejemplo: Comprobar si las palabras ab y a son aceptadas por el autómata probabilístico $A = (\{a,b\}, \{p,q\}, M, (0.5, 0.5), \{q\}, 0.5)$ donde:

$$M(a) = \begin{pmatrix} 0.5 & 0.5 \\ 0 & 1 \end{pmatrix} \quad M(b) = \begin{pmatrix} \cancel{1}^1_3 & \cancel{2}^2_3 \\ 1 & 0 \end{pmatrix}$$

ANÁLISIS LÉXICO

1. Análisis léxico

Todo compilador de lenguajes incluye un analizador de léxico. Éste se encarga de decidir si una palabra (*token*) pertenece o no al lenguaje.

Podemos construir un A.F.N.D. para cada una de las palabras reservadas del lenguaje, y otro, por ejemplo, para la detección de identificadores, basándonos en que empiece por letra, y el resto de símbolos deban ser letras o números.

Además, podríamos unir todos estos A.F.N.D. mediante un estado inicial que, mediante transiciones nulas, acceda a todos ellos, con lo que obtendríamos un ξ -A.F.N.D.

Finalmente podríamos pasarlo a A.F.D., e implementar un algoritmo que reconozca los tokens del lenguaje.